NXP — Engineering Learnings

Companion to the SoC Generation deck

ICLAD-DAC 2026 · GenAI Chip Hackathon

Harikrishnan KC · Team **Chip Convergence** · greatharikrishnan@gmail.com

1 · The 5-attempt live debug arc (~1 day, all fixes are tooling)

#	Failure class (real)	Deterministic fix	Result
1	5/8 specs missing required params; generator failures **silent**	typed generator-error feedback loop	3/8 IPs
2	model repeats omissions; emits `---` mega-block	**required-params schema auto-extracted from generator source**; multi-doc split	3/8
3	stitch **guesses port names** (`irq_sources` vs `irq_src`)	**module-interface index**: exact generated headers in the prompt	8/8, compile fail
4	wrong `default_div` (1 vs 26); IRQ bit-order swap	**library demo exemplars** + doc IRQ-map extraction	30/30, KAT 68/79
5	—	—	**PERFECT: 30/30 · 79/79 ×2 · STG 0-diff**

Every fix **auto-extracts from the provided materials** — nothing hand-tuned to this SoC.

The pattern: don't teach the model to stop guessing; **remove the need to guess.**

2 · Reset is the killer (and ports get "helpfully" renamed)

Published evidence we design against (SLDB, SpecAssess): frontier models **alter ports they were told not to touch**, and reset semantics is the **#1 silently-misspecified element** in spec-to-RTL flows.

We saw both live: the stitch renamed `irq_src` to `irq_sources` (attempt 3), and reset plumbing was the class of bug most likely to pass a shallow smoke test while corrupting everything downstream.

Responses: the port contract is *diffed token-level* against the TB skeleton (not reviewed by a model), and a dedicated **reset lint** enforces that `por_n` feeds only the synchronizer and that no reset net is split. Both are hard gates with typed errors.

3 · Model the library's bugs — don't fix them

We found the library's `apb_watchdog` **kick never reloads** the counter: `ctr <= ld1` is overridden by a later `ctr <= ctr - 1` in the same always block (last NBA wins).

The tempting fix — write a *correct* watchdog model — would be **wrong**: the hidden golden TB is built on the **same library**, so a first-principles model would fail candidates that are actually perfect. Our reference models are transcribed statement-for-statement, **quirks preserved**; the easy-8 calibrate to golden-recorded traces with 0 mismatches.

Lesson: an oracle's job is to predict the *evaluator*, not the ideal. Model reality.

4 · Some library output doesn't compile — auto-patch, provably neutral

`dma_engine` generates RTL where `cfg_rdata` is declared `output wire` but driven from `always @(*)` — an **elaboration error on every instance, for every team**.

Our agent auto-patches (`wire → reg`) at generation time. The patch is **behavior-neutral by construction** (same driver, legal declaration) and is applied only to *known, catalogued* library defects — never to model output.

Reported as a toolkit bug (one of **3 found**: this, the watchdog above, and the NVIDIA aes flow's masked failure — see the NVIDIA learnings deck).

5 · Mine the library's implicit knowledge

The correct UART `default_div` is **26**. That value appears **nowhere** in the architecture doc. The doc says 115200 baud @ 50 MHz — the *divider* is implied physics.

Attempt 4 failed on it (model guessed 1). The fix wasn't a hint in the prompt — it was extracting the library's **demo specs** (`rtl_gen_main.py --demo`) as exemplars: the demos encode the library authors' intended usage, including canonical derived values.

Lesson: provided materials contain more ground truth than their prose. Demos, generator source, TB skeletons — all machine-readable spec. Parse them; don't ask the model to recall.

6 · Verification layers must be *diverse*, not just deep

A FIFO_DEPTH 16→8 sabotage **passes the 30-check staged TB — 30/30.** Functional smoke tests exercise happy paths; capacity bugs hide in corners.

The KAT vector replay catches it in 5 trace lines: status read `0x89 vs 0x88` — literally the `tx_full` bit at the depth boundary.

And the third layer (STG random-stimulus differential) exists because *both* directed layers share blind spots by construction. Same principle as our NVIDIA stack: independent verification modes catch what redundant ones cannot.

7 · Sabotage-validate your gates (prove each fails on its own fault)

Every structural gate is validated by **planting the bug it claims to catch**: 8/8 sabotage suite (renamed ports, dangling IRQ nets, missing instances, fabric hang-offs, bridge-bus swaps…). The KAT oracle is validated both ways: it **fails** an inconsistent candidate (76/79) and **passes** a consistent variant design (79/79) — proving it follows the declared design rather than encoding one hardcoded answer.

The runner contract is tested end-to-end against a **mock of the contest's own HTTP protocol** (6/6), including retry/backoff behavior.

Meta-lesson (shared with our NVIDIA work): run everything before trusting it — *especially* the machinery that judges everything else.

Harikrishnan KC · Chip Convergence · greatharikrishnan@gmail.com